Helpmate AI V1.0

Github Repo:
https://github.com/ujwalabhishek/RAG-Generative-Search-HelpMate-Project

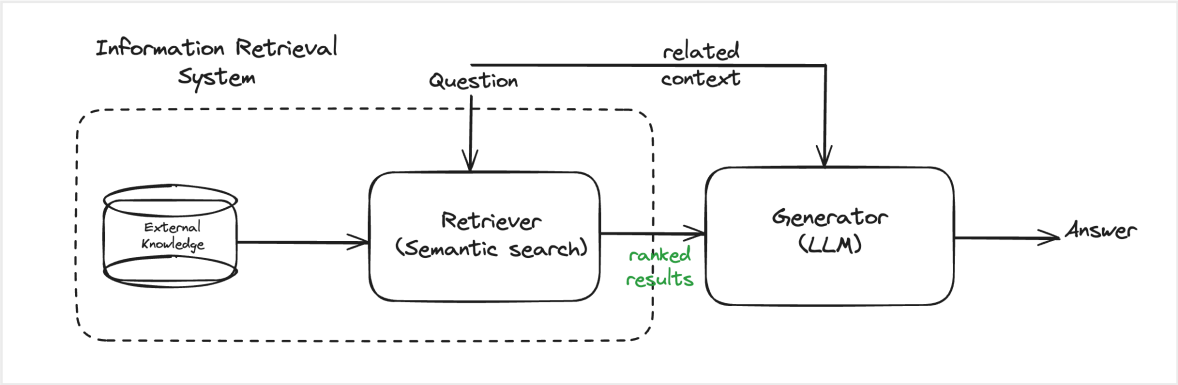
Objectives:

The goal of the project is to build a robust generative search system capable of effectively and accurately answering questions from a policy document.

Will be using a single long life insurance policy document for V1 of this project. The PDF for the document can be downloaded here Group Member Life Insurance Policy

Design:

Retrieval Augmented Generation (RAG) Pipeline



3-layer RAG: Embeddings, Search (cache + rerank), Generation (LLM).

The project would implement all the three layers effectively. It will be key to implement in order to build an effective search system.

1. **The Embedding Layer:** The PDF document needs to be effectively processed, cleaned, and chunked for the embeddings. Here, the choice of the chunking strategy will have a large impact on the final quality of the retrieved results. So, we will try out various strategies and compare their performances.

   Another important aspect in the embedding layer is the choice of the embedding model. We can choose to embed your chunks using the

OpenAI embedding model or any model from the SentenceTransformers library on HuggingFace.

2. **The Search Layer:** Here, we first need to design at least 3 queries against which we will test the system. We need to understand and skim through the document, and accordingly come up with some queries, the answers to which can be found in the policy document.

   Next, we need to embed the queries and search your ChromaDB vector database against each of these queries. Implementing a cache mechanism is also mandatory.

   Finally, we will implement the re-ranking block, and for this we can choose from a range of cross-encoding models on HuggingFace.

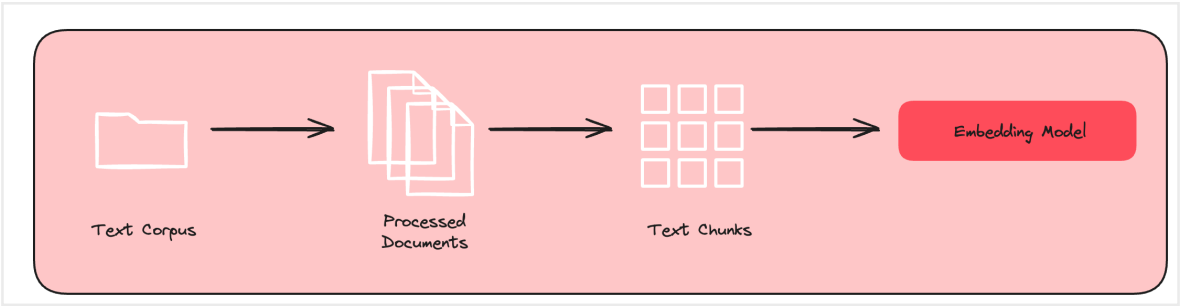
3. **The Generation Layer:** In the generation layer, the final prompt that you design is the major component. Make sure that the prompt is exhaustive in its instructions, and the relevant information is correctly passed to the prompt. You may also choose to provide some few-shot examples in an attempt to improve the LLM output.

Make sure you try out different strategies and models in each layer, and you might be surprised with the variety of top answers retrieved and generated by your system.

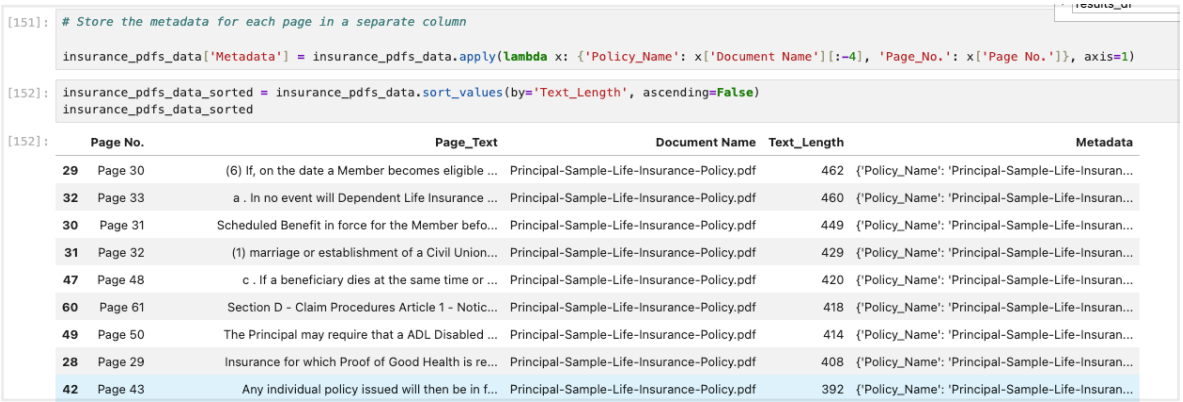
Implementations:

ChromaDB, embeddings, cache logic, cross-encoder reranking, prompt engineering.

1. **Text Processing:** In the first step, we process the documents (in this case, the documents pertain to the insurance domain) to extract the text, split it into smaller chunks and then pass them to the embedding model. The PDFPlumber library is used for processing the documents as it is very efficient in extracting the text contents of multiple PDF documents. The library can also represent a table in a neat list of lists format that preserves the original hierarchical structure of the document.



2. **Chunking & Embeddings:** We implemented the page-level chunking strategy to chunk the documents at a page level along with the metadata information.The reason behind this choice is primarily due to the structure of information in the insurance documents and the context window limit of the LLM model (GPT-3.5). Finally, you also append the relevant metadata information such as document name and page number for later retrieval.



```
[151]: # Store the metadata for each page in a separate column

insurance_pdfs_data['Metadata'] = insurance_pdfs_data.apply(lambda x: {'Policy_Name': x['Document Name'][:-4], 'Page_No.': x['Page No.']}, axis=1)
```

```
[152]: insurance_pdfs_data_sorted = insurance_pdfs_data.sort_values(by='Text_Length', ascending=False)
insurance_pdfs_data_sorted
```

[152]:

	Page No.	Page_Text	Document Name	Text_Length	Metadata
29	Page 30	(6) If, on the date a Member becomes eligible ...	Principal-Sample-Life-Insurance-Policy.pdf	462	{'Policy_Name': 'Principal-Sample-Life-Insuran...
32	Page 33	a . In no event will Dependent Life Insurance ...	Principal-Sample-Life-Insurance-Policy.pdf	460	{'Policy_Name': 'Principal-Sample-Life-Insuran...
30	Page 31	Scheduled Benefit in force for the Member befo...	Principal-Sample-Life-Insurance-Policy.pdf	449	{'Policy_Name': 'Principal-Sample-Life-Insuran...
31	Page 32	(1) marriage or establishment of a Civil Union...	Principal-Sample-Life-Insurance-Policy.pdf	429	{'Policy_Name': 'Principal-Sample-Life-Insuran...
47	Page 48	c . If a beneficiary dies at the same time or ...	Principal-Sample-Life-Insurance-Policy.pdf	420	{'Policy_Name': 'Principal-Sample-Life-Insuran...
60	Page 61	Section D - Claim Procedures Article 1 - Notic...	Principal-Sample-Life-Insurance-Policy.pdf	418	{'Policy_Name': 'Principal-Sample-Life-Insuran...
49	Page 50	The Principal may require that a ADL Disabled ...	Principal-Sample-Life-Insurance-Policy.pdf	414	{'Policy_Name': 'Principal-Sample-Life-Insuran...
28	Page 29	Insurance for which Proof of Good Health is re...	Principal-Sample-Life-Insurance-Policy.pdf	408	{'Policy_Name': 'Principal-Sample-Life-Insuran...
42	Page 43	Any individual policy issued will then be in f...	Principal-Sample-Life-Insurance-Policy.pdf	392	{'Policy_Name': 'Principal-Sample-Life-Insuran...

This concludes the chunking aspect also, as we can see that mostly the pages contain few hundred words, maximum going upto 460. So, we don't need to chunk the documents further; we can perform the embeddings on individual pages. This strategy makes sense for 2 reasons:

a. The way insurance documents are generally structured, you will not have a lot of extraneous information in a page, and all the text pieces in that page will likely be interrelated.

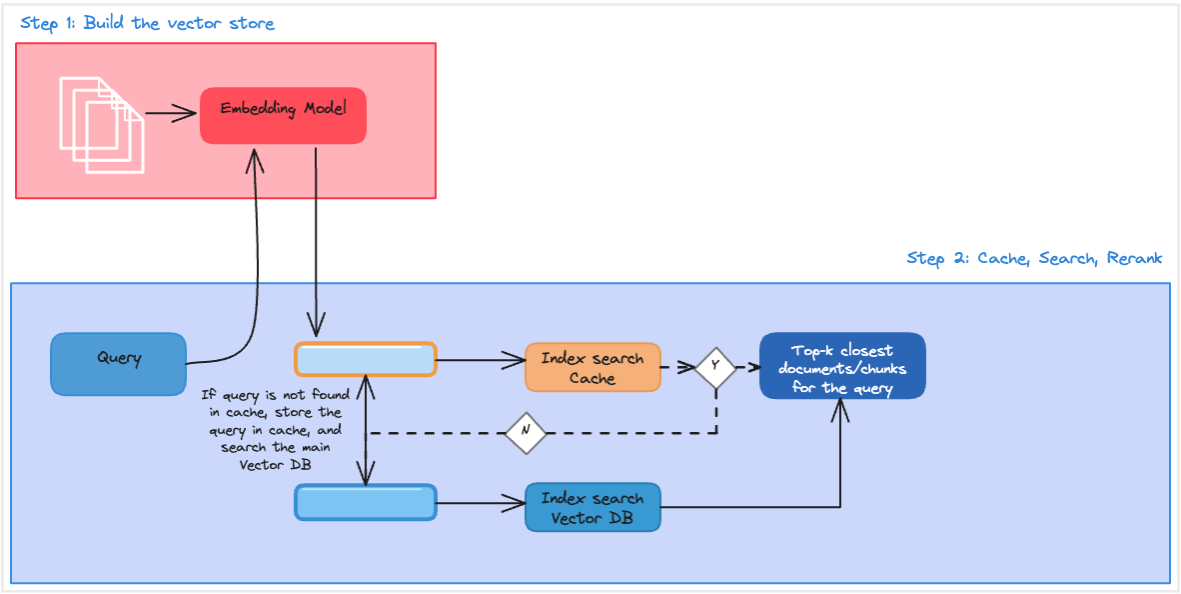
b. We want to have larger chunk sizes to be able to pass appropriate context to the LLM during the generation layer.

3. **Generate and Store Embeddings using OpenAI and ChromaDB:** we will embed the pages in the data-frame through OpenAI's text-embedding-ada-002 model, and store them in a ChromaDB collection. We need to first create the Chroma collections before we can start adding documents. The get_or_create_collections method, which will create a collection if not already present, and fetch it from your system if it has been created and stored previously. Next, since we are using

OpenAI embeddings and not Chroma's default embedding, we need to also pass the embedding function as an argument while creating the collection. Finally, the information that includes the document list, text and metadata information is passed to the chroma collection. Additionally, we also created a Chroma collection to serve as cache.

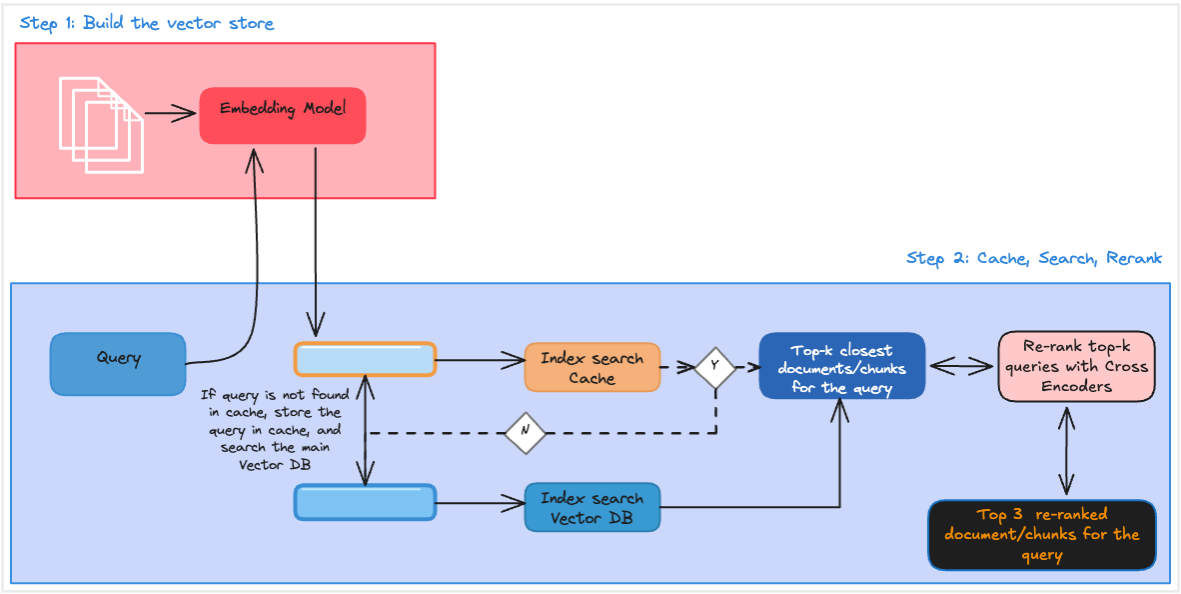
```
[31]: # Set the API key
      filepath = "../6.ShopAssist_ Data + Demo/"

      with open(filepath + "openai_key.txt", "r") as f:
        openai.api_key = ' '.join(f.readlines())

[32]: # Import the OpenAI Embedding Function into chroma

      from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

[33]: # Define the path where chroma collections will be stored

      chroma_data_path = './ChromaDB_Data'

[34]: import chromadb

[35]: # Call PersistentClient()

      client = chromadb.PersistentClient()

[36]: # Set up the embedding function using the OpenAI embedding model

      model = "text-embedding-ada-002"
      embedding_function = OpenAIEmbeddingFunction(api_key=openai.api_key, model_name=model)

[37]: # Initialise a collection in chroma and pass the embedding_function to it so that it used OpenAI embeddings to embed the documents

      insurance_collection = client.get_or_create_collection(name='RAG_on_Insurance', embedding_function=embedding_function)

[38]: # Convert the page text and metadata from your dataframe to lists to be able to pass it to chroma

      documents_list = insurance_pdfs_data["Page_Text"].tolist()
      metadata_list = insurance_pdfs_data['Metadata'].tolist()

[39]: # Add the documents and metadata to the collection alongwith generic integer IDs. You can also feed the metadata information as IDs by combining the policy name and page no.

      insurance_collection.add(
          documents= documents_list,
          ids = [str(i) for i in range(0, len(documents_list))],
          metadatas = metadata_list
      )
```

4. **Semantic Search with Cache:** The create a cache collection within the vector database that will try to cache the queries coming in and the corresponding responses. Creating a cache is important to preserve the scalability of the RAG system, particularly when documents span in the range of 1,000 or more and multiple users are using this application concurrently. With this additional layer, when a query is first input, the system first searches within the cache collection instead of the bigger collection. Cache implementation results in an improved response time from the system since a semantic similarity search need not be performed for a query that the system has already seen.



5. **Re-Ranking:** Another crucial element to improve the performance of the semantic search layer is re-ranking. The re-ranking stage improves the accuracy and relevance of semantic search results by assigning importance scores to the top K documents. Cross-encoder models, trained on labelled text pairs, are used to learn semantic similarity and improve search results.

There are a number of advantages of using cross-encoder models for re-ranking. First, these models can learn the semantic similarity between text sequences, which is a more accurate measure of relevance than traditional methods, such as keyword matching. Second, cross-encoder models can capture long-range dependencies in text, which can be important for understanding the meaning of a sentence or paragraph. Third, the models can be used to re-rank documents from any domain, without the need for any domain-specific knowledge.

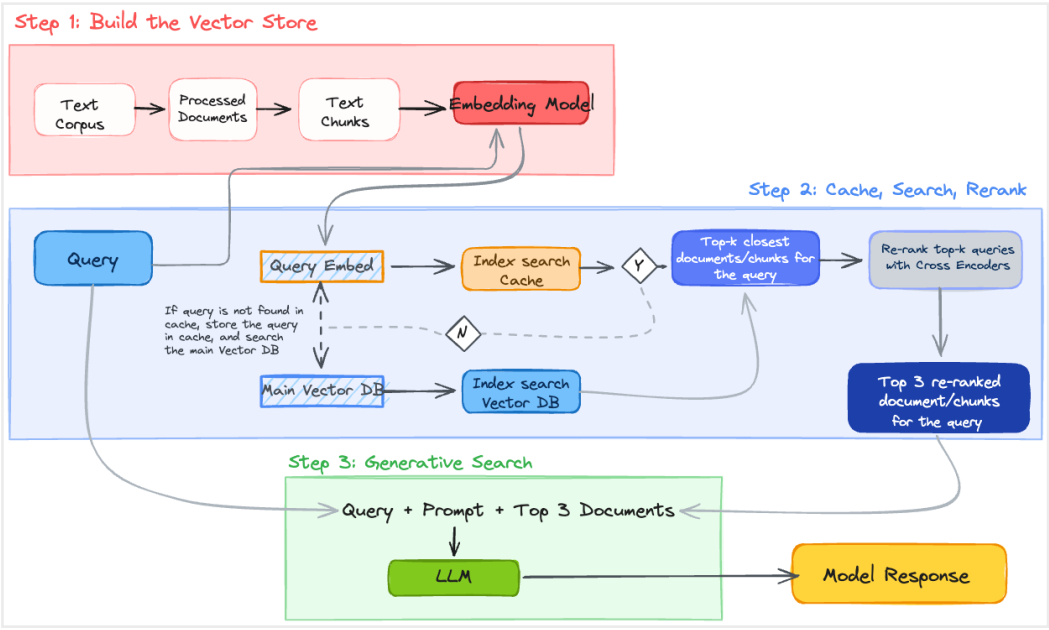


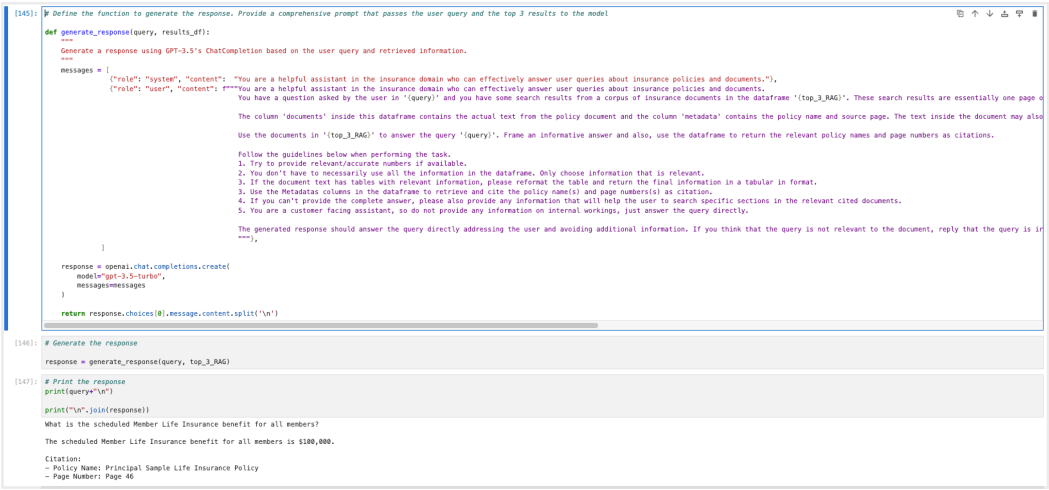

```
[137]: # Input (query, response) pairs for each of the top 20 responses received from the semantic search to the cross encoder
       # Generate the cross_encoder scores for these pairs

       cross_inputs = [[query, response] for response in results_df['Documents']]
       cross_rerank_scores = cross_encoder.predict(cross_inputs)

[138]: cross_rerank_scores

[138]: array([7.1206145 , 5.201272  , 1.9158204 , 1.9633144 , 5.455853  ,
              1.4470534 , 2.5913312 , 1.8580441 , 0.23288679], dtype=float32)

[139]: # Store the rerank_scores in results_df

       results_df['Reranked_scores'] = cross_rerank_scores

[140]: results_df
```

[140]:

	Metadatas	Documents	Distances	IDs	Reranked_scores
0	{'Page_No.': 'Page 46', 'Policy_Name': 'Princi...	PART IV - BENEFITS Section A - Member Life Ins...	0.214739	43	7.120615
1	{'Policy_Name': 'Principal-Sample-Life-Insuran...	Section C - Dependent Life Insurance Article 1...	0.246433	56	5.201272
2	{'Policy_Name': 'Principal-Sample-Life-Insuran...	Section A - Member Life Insurance Schedule of ...	0.254867	5	1.915820
3	{'Policy_Name': 'Principal-Sample-Life-Insuran...	(1) marriage or establishment of a Civil Union...	0.264602	29	1.963314
4	{'Policy_Name': 'Principal-Sample-Life-Insuran...	Section B - Member Accidental Death and Dismem...	0.264842	50	5.455853
5	{'Policy_Name': 'Principal-Sample-Life-Insuran...	b . on any date the definition of Member or De...	0.295206	18	1.447053
6	{'Page_No.': 'Page 54', 'Policy_Name': 'Princi...	f . claim requirements listed in PART IV, Sect...	0.296887	51	2.591331
7	{'Policy_Name': 'Principal-Sample-Life-Insuran...	Scheduled Benefit in force for the Member befo...	0.299255	28	1.858044
8	{'Policy_Name': 'Principal-Sample-Life-Insuran...	PART III - INDIVIDUAL REQUIREMENTS AND RIGHTS ...	0.303019	23	0.232887

6. **Generation Layer:** The next stage of the generative search application is the generation layer. This layer uses the generation capabilities of a large language model (LLM) to augment the system's output. The semantic search layer of our application performs the retrieval process or the information retrieval task and returns the top k documents for the user's query. These top k results, along with the user's query and the system prompt, are then passed to the LLM model to generate a model response that best answers the user's query. The LLM is provided with three inputs - query, prompt and top retrieved documents - as shown in the image below.





```
[145]: # Define the function to generate the response. Provide a comprehensive prompt that passes the user query and the top 3 results to the model

def generate_response(query, results_df):
    """
    Generate a response using GPT-3.5's ChatCompletion based on the user query and retrieved information.
    """
    messages = [
                {"role": "system", "content":  "You are a helpful assistant in the insurance domain who can effectively answer user queries about insurance policies and documents."},
                {"role": "user", "content": f"""You are a helpful assistant in the insurance domain who can effectively answer user queries about insurance policies and documents.
                                                You have a question asked by the user in '{query}' and you have some search results from a corpus of insurance documents in the dataframe '{top_3_RAG}'. These search results are essentially one page o

                                                The column 'documents' inside this dataframe contains the actual text from the policy document and the column 'metadata' contains the policy name and source page. The text inside the document may also

                                                Use the documents in '{top_3_RAG}' to answer the query '{query}'. Frame an informative answer and also, use the dataframe to return the relevant policy names and page numbers as citations.

                                                Follow the guidelines below when performing the task.
                                                1. Try to provide relevant/accurate numbers if available.
                                                2. You don't have to necessarily use all the information in the dataframe. Only choose information that is relevant.
                                                3. If the document text has tables with relevant information, please reformat the table and return the final information in a tabular in format.
                                                3. Use the Metadatas columns in the dataframe to retrieve and cite the policy name(s) and page numbers(s) as citation.
                                                4. If you can't provide the complete answer, please also provide any information that will help the user to search specific sections in the relevant cited documents.
                                                5. You are a customer facing assistant, so do not provide any information on internal workings, just answer the query directly.

                                                The generated response should answer the query directly addressing the user and avoiding additional information. If you think that the query is not relevant to the document, reply that the query is ir
                                                """},
            ]

    response = openai.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=messages
    )

    return response.choices[0].message.content.split('\n')

[146]: # Generate the response

response = generate_response(query, top_3_RAG)

[147]: # Print the response
print(query+"\n")

print("\n".join(response))

What is the scheduled Member Life Insurance benefit for all members?

The scheduled Member Life Insurance benefit for all members is $100,000.

Citation:
- Policy Name: Principal Sample Life Insurance Policy
- Page Number: Page 46
```

Challenges:

Latency vs Accuracy:
In the context of our generative search system for the long life insurance policy document, there is often a trade-off between latency and accuracy. Lower latency means the system can respond faster to user queries, which is important for usability, especially in a live setting with multiple users. However, maximising accuracy typically involves deeper semantic searches, cross-encoder re-ranking, and thorough prompt conditioning, which can increase processing time. For example, retrieving top results from the ChromaDB with cache might be fast, but if we always re-rank with a heavy model, response time will rise. Balancing these two aspects is essential to ensure a smooth user experience without compromising too much on the quality of the answers.

Chunk Tuning:
Chunk tuning refers to deciding how to split the life insurance policy PDF into

optimal units of text for embedding. In this project, we observed that most policy pages had a few hundred words, with a maximum of about 460 words per page. We used page-level chunking with metadata (document name and page number) to preserve context. Fine-tuning chunk size is critical for two reasons: too small a chunk might spread context thin and lead to LLM hallucinations, while too big a chunk might exceed the LLM context window, reducing retrieval effectiveness. Adjusting the chunk size according to the structure of the insurance policy ensures that the system retrieves meaningful and context-rich data for accurate responses.

Hallucination:
Hallucination occurs when the LLM generates content that is not grounded in the retrieved policy document, often because of weak retrieval signals or insufficient context in the prompt. In our project, hallucinations can mislead users by presenting inaccurate policy details. To reduce hallucination, we ensure that the top K results from the search layer are meaningful and that the generation prompt explicitly instructs the LLM to stick to the retrieved text. Additional measures, like re-ranking with cross-encoders and providing metadata context, further help the model generate grounded and accurate answers to insurance-related queries.

Lessons Learnt:
Retrieval quality + prompt design are critical for RAG